

voxelcraft — audit: errors found in the code and the documentation

Scope: the whole tree as reconstructed from the uploaded dump (169 files, `/home/user/repo`). Method: full read of the briefing/roadmap/ledger/errors/perf docs, mechanical cross-checks (doc → code identifier and constant resolution, flag/bit maps, markdown table shape, link and anchor resolution), and an actual build + test run with the local Rust 1.98.1 toolchain.

Environment caveat, because two findings depend on it: this box has **2 cores / 1 GB RAM** and no GPU. Anything needing a device (`cargo run`, `--bin harness`, screenshots) cannot run here; compile-time and CPU-only tests can, and did.

Reproduce:

```
cd /home/user/repo && export PATH=$HOME/.cargo/bin:$PATH
cargo check --offline --all-targets          # clean, no errors
cargo test  --offline --no-fail-fast         # 372 pass, 10 fail, 2 targets red
```

Summary of findings

#	Kind	Where	Confidence
F1	Batch 93's probe-lattice change (8 → 4 blocks) never reached the prose: 15 statements still describe the old field	8 code sites, 4 docs, 3 in <code>CLAUDE.md</code>	certain
F2	VRAM bookkeeping now disagrees with itself across three files	<code>CLAUDE.md:505</code> , <code>docs/roadmap.md:1027-1029</code> , <code>PERF.md:337</code>	high
F3	10 tests fail; <code>HANDOFF.md</code> claims 48/48 targets pass	<code>HANDOFF.md:20</code> , <code>tests/lod_stream.rs:28</code> , <code>tests/shadow_grid.rs:36</code>	certain (reproduced)
F4	Soft shadows: dead shader constants, an orphaned comment, and docs describing a mechanism the shader no longer has	<code>common.wgsl:2199/2210/2263</code> , <code>mod.rs:760</code> , <code>CLAUDE.md:156</code> , <code>tests/batch102.rs:6-10</code>	high
F5	Stale counts in code comments (<code>four</code> pass files; <code>three</code> / <code>five</code> offset asserts)	<code>mod.rs:3673</code> , <code>mod.rs:1333</code> , <code>CLAUDE.md:339-340</code>	certain
F6	Markdown tables broken by unescaped pipes / rows detached from their table	<code>CLAUDE.md:148</code> , <code>docs/ledger.md:86</code> , <code>docs/ledger.md:135</code> , <code>docs/roadmap.md:83</code> , <code>docs/ledger.md:46/48/50</code>	certain
F7	<code>src/bin/*.rs</code> is absent although 17 references expect it, and <code>tests/bins.rs</code> silently no-ops instead of failing	<code>Cargo.toml:5</code> , <code>tests/bins.rs:10-20</code>	medium (see caveat)
F8	<code>PERF.md</code> and <code>docs/ledger.md</code> point at <code>tests/pretest.rs</code> , which is not in the tree	<code>PERF.md:1300</code> , <code>docs/ledger.md:74</code>	medium

F1 — The probe lattice's prose was left at 8 blocks after L5 shipped 4

Established fact (the authority): `src/probe.rs:107` `pub const SPACING: i32 = 4;`, with `src/render/mod.rs:3568/3572` computing `PROBE_DIM_XZ = 512 / SPACING` and `PROBE_DIM_Y = WORLD_HEIGHT / SPACING` and pinning both to **128** by `const _: () = assert!;` `src/render/shaders/resolve.wgsl:285` `const PROBE_SPACING: f32 = 4.0;`; `docs/roadmap.md:1023` **"Shipped as the constants that define it** (commit `589093a`): `SPACING 8 -> 4`".

Statements that still say 8 (or are derived from 8):

file:line	what it says	correct value now
<code>src/probe.rs:1</code>	module doc: "a six-axis ambient cube every 8 blocks "	4
<code>src/probe.rs:43</code> <code>7-438</code>	"shared by the eight probes stacked in it ... the difference between 20,480 height samples per chunk and eight times as many"	16 probes, 81,920 samples (256 columns × 16 azimuths × 20 steps), 16×
<code>src/probe.rs:69</code> <code>3</code>	"the chunk's 20,480 height samples stay 20,480"	81,920
<code>src/stream.rs:2</code> <code>2</code>	"a probe eight blocks inside a chunk of pure air is still read by the trilinear tap"	four blocks
<code>src/render/mod.rs:1827</code>	" <code>linear_sampler</code> clamps ... an eight-block band of wrong shading on one plane every 512 blocks"	four-block band (period 512 is still right)
<code>src/render/mod.rs:3508</code>	"Y is exact rather than toroidal because <code>WORLD_HEIGHT / SPACING</code> is 64 on the nose"	128 — the same doc block declares <code>PROBE_DIM_Y == 128</code> three lines below
<code>src/render/mod.rs:3516</code>	"a shading factor, eight blocks wide , on a chunk behind the camera"	four blocks wide
<code>src/render/shaders/common.wgsl:218</code>	"would put an eight-block band of wrong shading on one plane every 512 blocks"	four-block band
<code>docs/shading.md:486</code>	"The tap is taken at <code>hit + n * PROBE_SPACING * 0.5</code> , four blocks along the surface normal"	two blocks (<code>PROBE_NORMAL_BIAS = PROBE_SPACING * 0.5 = 2.0</code> , <code>resolve.wgsl:333</code>)
<code>docs/shading.md:489-490</code>	"the lattice is 8 blocks coarse and a one-block nudge leaves seven eighths of the weight where it was"	four blocks coarse, three quarters
<code>docs/adr/0002-no-disc-sampled-sun.md:114</code>	"the envelope wins on every axis — <code>TEXEL = 4</code> blocks against <code>SPACING = 8</code> "	the comparison is 4 against 4 — no longer a win (ADR, so arguably frozen, but the number is false for the shipped build)
<code>CLAUDE.md:146</code>	<code>--no-probe-cube</code> row: " 8 blocks per probe "	per 4 blocks
<code>CLAUDE.md:392</code>	cross-file invariants: "one probe per 8 blocks "	one per 4 blocks
<code>CLAUDE.md:505</code>	machine table: " probe field 12.0 ... 64 x 384 x 64 <code>Rgba16Float</code> is exactly 12,582,912 bytes ... <i>derived rather than sampled</i> and so cannot drift"	100.7 MB, 128 x 768 x 128 — it drifted, in the one row that claims it cannot
<code>CLAUDE.md:623-624</code>	"the probe lattice — 384 in its largest axis — has forty times the headroom"	largest axis 768 (<code>PROBE_DIM_Y * AXES = 768</code>), ≈21x headroom

Note `docs/ledger.md:84` ("every 8 blocks", row 57) and `docs/ledger.md:117` ("SPACING 8 costs 3,256 us", row 93) are **historical records** of what those batches measured and are *not* errors; I list them only so the next reader does not "fix" them.

F2 — The VRAM totals contradict each other

- `CLAUDE.md:505`: "screen buffers 66.4 MB, pairs 8.4, shaft 3, world data ~50, **probe field 12.0 — ~142 MB ... 96.5% free**". $66.4 + 8.4 + 3 + 50 + 12 \approx 140$ MB, so 96.5% of 3.9 GB is derived from the **old** 12 MB field.
- `docs/roadmap.md:1027-1029`: "12.6 MB -> **~100.7 MB** of the 3.9 GB of VRAM, **which is 96.5% free**". The 96.5% is borrowed from the old total; with the shipped field the total is ≈ 230 MB → $\approx 94\%$ free. The two documents cannot both be right.
- `PERF.md:337`: "**VRAM: 12 MB**, one 64 x 384 x 64 `Rgba16Float` 3D texture ... **24 KB** of it per resident chunk" — the per-chunk share is now 192 KB ($96 \text{ MiB} \div 512 \text{ chunks}$ covering the $512 \times 512 \times 512$ field). This one sits in a dated batch-57 section, so treat it as stale-in-context rather than as a live claim.

`src/probe.rs:106` and `src/render/mod.rs:3575` both carry the correct post-L5 numbers ("~100 MB", "~100.7 MB"), so the code side is consistent; the briefing and the roadmap are not.

F3 — The test suite is not green, and the handoff says it is

`HANDOFF.md:20` (row CL02): "**All 48 test targets pass release execution (100%)**".
Actual run here (dev and release profiles agree):

```
372 tests passed; 10 failed; 48 targets; 2 targets failed
  `--test lod_stream` : 3 passed; 6 failed
  `--test shadow_grid` : 0 passed; 4 failed
```

Every failure is the same panic, from the `settle()` helpers at `tests/lod_stream.rs:28-37` and `tests/s/shadow_grid.rs:36-45`:

```
for _ in 0..6000 {
    m.update(cam, w, Duration::from_millis(50));
    if m.is_settled(w) { return; }
    std::thread::sleep(Duration::from_micros(200));
}
panic!("world never settled");
```

I resolved this rather than leaving it as a maybe, by running the same configuration in a scratch copy outside the repo with a longer budget:

```
i=5000    t=1.73 s   in_flight=64   resident=444/760
i=15000   t=4.99 s   in_flight=64   resident=585/760
SETTLED   i=24904   t=8.26 s   chunks=760
```

So: not a hang, not an infinite request loop — the world does settle, it just needs $\approx 25,000$ `update()` calls on 2 cores where the helper allows 6,000 (≈ 1.2 s of sleep with a 50 ms frame budget). The budget is a machine-speed assumption with no wall-clock deadline, so the failure is reproducible here and would not appear on the 16-thread machine the docs' timings assume. Both a claim error in `HANDOFF.md` and a portability defect in the two test files.

Related, same file: `HANDOFF.md:6` states "***No build, test or benchmark was run during this closeout**", which is hard to reconcile with the CL02 row's 100%-pass claim in the same document.

F4 — Soft shadows: constants nothing reads, a comment with no body, docs describing the old arm

The shipped penumbra is the screen-space reconstruction:

- `src/render/shaders/shadow.wgsl:1-3` — "Phase 1/2 **screen-space** penumbra reconstruction. Not a multi-ray area-light estimate."
- `shadow.wgsl:44` — "**EXACTLY ONE** geometric shadow query for an eligible receiver."
- `src/config.rs:1133` — help text: "Primary shadows always use one ray + optional screen-space reconstruction".
- `BATCH-LOG.md:100-112` (DS02) — the rewrite, including "The old `SOFT_TAPS` compatibility override may still be present in the source/table but **no longer drives a ray loop**."

What is still written as if the three-cone-tap arm were the live wiring:

- `src/render/shaders/common.wgsl:2199` `override SOFT_TAPS: u32 = 3u;` — set by `make_spec` (`src/render/mod.rs:4060`, 7.0/3.0 under `FLAG_HI_SOFT_HQ`) and `**read by no shader code anywhere**`.
- `src/render/shaders/common.wgsl:2210` `const SOFT_MISS_DIST: f32 = 64.0;` — referenced by nothing at all.
- `src/render/shaders/common.wgsl:2190-2197` describes a fold that does not exist: `"*o is the one-constant revert* (with the arm still armed: the fold below returns the centre bit and the loop never compiles a tap)"`.
- `src/render/shaders/common.wgsl:2263` — a comment introducing "The penumbral verdict: the centre bit plus `SOFT_TAPS` cone taps, evenly weighted" — followed by two blank lines and `// ---- water ----`. The function it documents has no body.

- `src/render/mod.rs:760` and `CLAUDE.md:156` still describe `--soft-shadows` as "three cone taps around the centre shadow ray" with its +4.545 ms cost story, without noting that the flag now arms `shadow.wgsl`'s bilateral filter instead.
- `tests/batch102.rs:6-10` — "the arm is **three extra rays through the occlusion grid**, so every number that bounds those rays (tap count, cone aperture, miss-budget) is pinned as a literal next to its revert". By the same file's own standard one test earlier ("an override left at its default is a flag that does nothing in silence"), `**SOFT_TAPS` is now exactly such an override`**`: parsed, masked, plumbed into `make_spec`, and inert.

Note `SOFT_PENUMBRA` is live, but for a different job (a screen-space radius multiplier, `shadow.wgsl:54`), so its doc block at `common.wgsl:2180` — "the tap cone's half-angle" — is also describing the retired mechanism.

F5 — Stale counts in code comments

- `src/render/mod.rs:3673`: "The one module every compute pass is compiled from: `common.wgsl` and the **four pass files**, concatenated in this order and no other." — `shader_source()` (lines 3683-3689) concatenates **seven** files (common + tile_select, march, resolve, taa, shaft, shadow).
- `src/render/mod.rs:1333-1334`: "**These three** pin the two seams batches 10 and 11 actually moved." — there are **seven** `offset_of!` asserts directly below it (lines 1335-1341: *cloudcover 192*, *godraystrength 208*, *tintstrength 224*, *waveamp 240*, *shaft_origin 256*, *cloudpatch 272*, *prevview_proj 288*), and the history block right above records **six** append batches (10, 11, 12, 16, 35, 95).
- `CLAUDE.md:339-340`: "**Five batches** appended **four** words before `prev_view_proj` ... and **five** `offset_of!` asserts pin the seams" — six batches, seven asserts.

F6 — Markdown that does not render as written

Unescaped pipes split table rows into extra columns:

- `CLAUDE.md:148` (`--no-light-rgb` row) contains "`sky:4 | r:4 | g:4 | b:4`" — the row has 6 cells where the table has 3.
- `docs/ledger.md:86` (row 59) contains the same string — 7 cells where the table has 4.
- `docs/ledger.md:135` (row 102-driver-gate) contains "VULKAN|DX12" unescaped — breaks that row.

Rows detached from their table:

- `docs/roadmap.md:83` — the **D1** row sits after the closed-designation table has been closed by

a blank line (82) and before the next paragraph (85), so it renders as a separate, headerless one-row table. It is also the one entry in that list that says "shipped" rather than recording where a designation went — the row was presumably appended after the table ended.

- `docs/ledger.md:46`, `:48`, `:50` — rows 28, 28b and 29 are each alone between blank lines, so they render as three headerless one-row tables *inside* the batch timeline.

The tree's own guard (`tests/docs.rs`) checks links, flag presence and README budgets; it has no opinion on table shape, which is why all of the above is invisible to `cargo test`.

F7 — `src/bin/` is referenced 17 times and is not in the tree

`Cargo.toml:5` says "Two binaries live here (`voxelcraft` and `probe`)"; `README.md:80-90` and `CLAUDE.md:79-88` document three (`probe`, `shaderstats`, `harness`) with example commands; `tests/bins.rs`, `docs/harness.md`, `docs/ledger.md` and `PERF.md` all name `src/bin/harness.rs`, `src/bin/probe.rs`, `src/bin/shaderstats.rs`. The directory does not exist in this tree.

What makes it worth reporting rather than shrugging at is the guard's behaviour:

```
fn source(rel: &str) -> Option<String> {
    let p = Path::new(env!("CARGO_MANIFEST_DIR")).join(rel);
    if !p.exists() { return None; }           // tests/bins.rs:10-14
    ...
}

let Some(h) = source("src/bin/harness.rs") else { return; }; // tests/bins.rs:20
```

All four `tests/bins.rs` tests pass by early-returning, so the absence of the binary trilogy is invisible to the suite — the guard against a stale harness vocabulary currently asserts nothing.

Caveat: the uploaded dump may simply not have included `src/bin/*.rs` (it also cannot be checked for the `docs/research/` material many docs link to). If the source tree is meant to be complete, this is a real gap; if not, the finding is only about the silent skip.

F8 — Two documents cite a test file that does not exist

- `PERF.md:1300`: "the checks that matter here are **five source-level guards** in `tests/pretest.rs` and one paired bench" — and `PERF.md:1335-1342` still tabulates `SPEC_VIS_PRETEST=1/0` and `--no-vis-pretest`.
- `docs/ledger.md:74` (row 50) records the same file.

`pretest` survives in exactly three places in the tree: `src/render/shaders/march.wgsl:97-108` (which says the arm "was built, measured and thrown away in batch 50. **Do not rebuild it**"), `docs/errors.md:932`, and the two docs above. There is no `tests/pretest.rs`, no `--no-vis-pretest` flag in `src/co`

`nfig.rs`, and no `SPEC_VIS_PRETEST` override in any shader. The features being described are correctly documented as *unshipped* elsewhere, so the stale part is specifically the present-tense reference to a file and a flag that no longer exist — the same drift class the `docs.rs` guard was written for, one level down (backticked paths are not links, so nothing checks them).

Checked and found clean (so these are not findings)

- **Links and anchors:** all markdown links in the tree resolve except the documented exemptions (`screenshots/*.png`, `src/bin/*.rs`, `docs/research/`, the two look-reference images), and every in-page `#anchor` resolves (new sweep — 0 failures).
- **Flags:** every flag the parser accepts appears in `--help`; every flag in `CLAUDE.md`'s control table is accepted (including the `--no-` forms); `README.md` is inside its 20-flag budget; no document transcribes a test count; no README measurement collides with `PERF.md`.
- **Flag bits:** the 24 `FLAG_*` constants mirrored into `common.wgsl:244-267` are bit-for-bit identical to `src/render/mod.rs`; word 2 (`FLAG_HI_*`, `FLAG_PROBE_*`) is a clean power-of-two run with the documented aliases; `FLAG_SKY_TINT` really is bit 31 and `FLAG_PROBE_SUN_HIGH` bit 30.
- **Overrides:** all 51 `override`s declared in the shaders are supplied by `make_spec`, and every `"SPEC_..."` string in `src/render/mod.rs` names a declared override — no orphans in either direction.
- **GpuFrame:** Rust `size_of == 352` and the seven `offset_of!` values (192/208/224/240/256/272/288) match the WGSL `Frame` struct exactly, field order included.
- **Build:** `cargo check --all-targets` is clean; no compile errors or warnings surfaced.
- **The suspicious-looking sea level is correct:** `src/worldgen.rs:12` `SEA_LEVEL = 128` with the comment "(world y = 64)" reads like a contradiction, but `src/math.rs:2` defines "displayed Y is internal - 64" and `docs/terrain.md:108` says "128 internal (displayed 64)" — consistent. Likewise `src/shaft.rs` `SOFT = 4.0` vs its "about 3.7 blocks at 400" is the sun's own disc, and `PAIRS_PER_TILE = 16` vs "worst observed 10.1" is a budget over an observation.
- **Constants quoted in prose that I could verify mechanically** agree with the code:
`LEAF_FILL 0.62`, `GROUND_F0 0.04`, `WATER_FAR_DIST 128.0`, `MEADOW_SPREAD 1.75`, `G2_TALL_S HARE 24`, `PROBE_DIM_XZ 128`, `UPLOAD_REACH 224.0`, `SPACING 4`.
- **Documented commands parse:** every `--flag` inside a fenced block in every `.md` is either a `voxelcraft` flag the parser accepts, a `cargo` flag, or a `harness` subcommand flag (the harness lives in the missing `src/bin/`, see F7).

Not investigated (out of reach here)

- Anything requiring a device: screenshot/vantage work, `bitexact`, `bench-frames`, the `validation/*.py` diagnostics (they need the `wgpu` / `tree_sitter` Python packages, which are documented as

not shipped).

- Re-measurement of any quoted timing, MAE or pixel count — only internal arithmetic was checked.